

Sim2Sim before Sim2Real

Two mappings, one encoder, and the single difference between the two problems

The page opens by conceding the thing the field usually argues about.

“An agent that learns to attack or defend a network learns it somewhere. That somewhere is a simulator, because letting a learning agent explore production infrastructure by trial and error is not an option.”

website, transfer-to-realistic-environments.md, opening

So the question is not whether to simulate. It is what happens next, and the position this project takes is that sim-to-real and sim-to-sim are one problem with one difference between them. That position is the first entry below, because everything else in the file is built on it.

Two warnings about the vocabulary before the entries. First, the object that moves here is an offensive policy across simulators, where the neighbouring project moves a defensive policy across two backends of one testbed. The word transfer carries a different noun in each. Second, the method’s two symbols are overloaded in the manuscript, and the entry for each says how.

Contents

1	Active	2
1.1	The problem	2
1.2	The two mappings	5
1.3	The encoder	7
1.4	The evaluation	10
1.5	The environments	13
2	Working	14
3	Peripheral	15
4	The clashes, listed	16
5	What crosses into other files	17
6	Sources	17

1 Active

1.1 The problem

sim2real

“Sim2real refers to the long standing challenge encountered by embodied agents, between where a policy is trained and where it would run.”

website, toward-deployment/README.md

Definition: The distance between the environment a policy was fitted in and the environment it has to act in, when the second is the deployment itself. Dynamics and sensing differ, the deployment ages away from the data, and any safety claim has to hold while the model is wrong.

Distinguishes it from: Sim2sim, by the target only. The three sentences after the definition on that page are the three things that make the target hard to know, and knowing the target is the whole of the difference.

Operational test: Is the target the place the policy is meant to run, or a stand-in for it? If yes, uncertainty about the target is part of the problem statement rather than an experimental inconvenience.

In my project: The heading of the Toward Deployment overview, and the reason this project exists as the third of three answers to it.

sim2sim

“One key insight we had is that the sim2real challenge can be transformed to a sim2sim without loss of its complexity.”

website, toward-deployment/README.md

Definition: Moving a policy between two fully specified simulators whose state spaces, action vocabularies and observation models do not coincide. The target is known, and the problem is still there.

Distinguishes it from: Sim2real, which adds only the uncertainty about the target. And from generalization across configurations of one simulator, where no translator has to be built because the vocabulary is already shared.

Operational test: Does a wrapper have to be written before the policy can emit a legal action in the target? If yes, this is the regime.

In my project: The reason the whole evaluation runs between simulators. The claim that no complexity is lost in the transformation is what licenses it, and that claim is the next entry.

one problem, one difference

“We view sim-to-sim and sim-to-real transfer as instances of the same underlying problem: aligning policies across environments with different state and action representations.”

RAISE, 1_introduction.tex

“Sim-to-real is the same problem as sim-to-sim, with one extra difficulty: the target is only partially

known.”

website, transfer-to-realistic-environments.md

Definition: The project’s central position. The two settings are not neighbours on a difficulty scale. They are the same alignment problem, and sim-to-real carries one additional unknown.

Distinguishes it from: The claim that sim2sim is easier. The page refuses that explicitly: supplying the missing ground truth collapses sim-to-real back into sim-to-sim, which is still hard, because finding the abstraction that relates two fully specified environments is itself computationally difficult.

Operational test: Take a proposed transfer mechanism and ask whether supplying full knowledge of the target would make it unnecessary. If it would, the mechanism was addressing the uncertainty and not the alignment.

In my project: It is the argument for the whole experimental design. A mechanism that works across two simulators is being offered as a mechanism that will work into a deployment, and the sentence above is the warrant.

the two use cases

“An organization trains a red-teaming agent in simulation and wants to run it against its own network.”

website, transfer-to-realistic-environments.md

“An organization wants to compare its agent against one trained in a different simulator.”

website, transfer-to-realistic-environments.md

Definition: Deployment and comparison, given as the two places the problem shows up. The page’s move is to say they look separate and are not, because both reduce to aligning policies across environments that speak different representations.

Distinguishes it from: A motivation paragraph. These two are the reason the same mechanism has to serve both, and the second one is what makes the work a benchmarking contribution as well as a deployment one.

Operational test: Name who wants the transfer and what they get from it. An operator wants the first. A reviewer wants the second.

In my project: Both are in the introduction, and the related work section says why the second is currently impossible: the field is siloed per simulator.

siloed per simulator

“despite best efforts to conform to similar abstractions, the simulators are inevitably tightly coupled with their backend design”

RAISE, 2__relatedwork.tex

“As a result, the field is siloed per simulator. Results do not compare across environments, and an agent or interface built for one does not carry to another.”

RAISE, 2__relatedwork.tex

Definition: The state of the field this work is answering. The differences in backend produce distributional differences even in the features two simulators hold in common, such as observation dimensionality and action granularity.

Distinguishes it from: A standards complaint. The claim is not that nobody agreed on an abstraction. It is that agreeing on one is not enough, because the implementation shows through the abstraction anyway.

Operational test: Take a feature two simulators both claim to expose. Do its values have the same distribution? If not, the abstraction is shared and the numbers are not.

In my project: It is why state alignment needs a fitted stage after the deterministic one, which is the single most consequential design decision in the method.

cross-domain transfer

“We face one of the harder cases in policy transfer with mismatch action and observation space, which places us in the cross-domain transfer regime”

RAISE, 2_relatedwork.tex

Definition: The transfer regime in which neither the state space nor the action space is shared between source and target, so no part of the policy can be applied without a constructed correspondence.

Distinguishes it from: The same-domain case, where a policy can at least be run and merely performs badly. Here it cannot emit a legal action at all until something is built.

Operational test: Can the source policy’s output be passed to the target without a wrapper? If not, the case is cross-domain and the wrapper is part of the method rather than part of the test rig.

In my project: The regime label the related work section uses to place the work, and the reason domain-adversarial alignment is the family the method draws from.

zero-shot transfer

“Can offensive cyber policies transfer zero-shot across environments, with no retraining?”

website, transfer-to-realistic-environments.md, question 1

Definition: Executing the source policy in the target with no gradient step taken in the target at any point. The encoder is pretrained offline and frozen, and the policy is untouched.

Distinguishes it from: Fine-tuning, few-shot adaptation, and anything that consumes target rewards. The pretraining requirement here is observations from a random policy, which is a weaker ask than any of those.

Operational test: Did any parameter change while the target was being interacted with? One yes and the result is not zero-shot.

In my project: The first of the three questions, and the condition every number in the results tables was produced under.

the domain gap

Definition: The distance between source and target, held as the independent variable of the

experiment. One source is held fixed and two targets are chosen at different distances from it, so that representational distance and intrinsic task difficulty can be separated as factors.

Distinguishes it from: Task difficulty, which the design is built to hold apart from it. Net-SecGame is the harder task and the smaller gap. CyberBattleSim is the easier task and the larger gap. Confounding the two would make either result unreadable.

Operational test: Ask what varies across the two target environments. If only difficulty varies, the design measures difficulty and not gap.

In my project: The rationale for the environment selection, and the reason the two result tables rank the conditions in opposite order.

1.2 The two mappings

split transfer into two mappings

“Split transfer into two mappings and build each without touching the underlying simulators.”

website, transfer-to-realistic-environments.md

Definition: The method’s shape. State and action are treated as separate sub-problems and solved separately, and neither solution requires a change to either simulator.

Distinguishes it from: A shared-interface proposal, which asks the simulators to conform. This builds outward from each simulator instead, so adding a third environment is a wrapper and not a negotiation.

Operational test: Would the method require a pull request against the target simulator? If yes, it is a standard and not a transfer mechanism.

In my project: Stated on the page in one line and formalized in the manuscript as the state mapping and the action mapping.

action translation, ψ

“A shared, deterministic action vocabulary in which each action advances a chosen host along the kill chain.”

website, transfer-to-realistic-environments.md

“Wrapper logic resolves the choice into whatever the target simulator natively expects.”

website, transfer-to-realistic-environments.md

Definition: A mapping from the shared action space into the target’s native action space. The shared space contains one action per host, meaning advance that host along the kill chain, plus a no-op.

Distinguishes it from: An action embedding. Nothing is fitted. The mapping is a table, and the manuscript prints it: one row per kill-chain phase, one column per target, giving the native call each phase resolves to.

Operational test: Can the mapping be written down before any data is collected? If yes, it is this. If it needs trajectories, it is something else.

In my project: Built for three target pairs. Where a target has no native equivalent for a phase, the mapping for that phase is constant and the host choice has no effect, which the table says row by row.

the policy’s intent

“The policy’s intent, which host to move on next, is decoupled from the mechanics of expressing it.”

website, transfer-to-realistic-environments.md

Definition: What the shared action space is chosen to carry. The decision being transferred is target selection, not tool selection, and target selection is the part that means the same thing in every environment.

Distinguishes it from: The native action, which is simulator-specific and which the policy never sees. This is the reason the split is between intent and mechanics rather than between two vocabularies of equal standing.

Operational test: Ask whether the decision could be stated to a human operator without naming a tool. If yes, it is intent.

In my project: It is why the shared action space is one action per host rather than one action per technique, and it is the assumption the whole translation layer rests on.

state alignment, ϕ

“Each raw observation is projected into a fixed-size vector organized by kill-chain stage per host: phase, reachability, attacker presence, target designation, plus a small block of episode-level context.”

website, transfer-to-realistic-environments.md

“This is computed deterministically per environment, with nothing learned.”

website, transfer-to-realistic-environments.md

Definition: A per-environment wrapper that rewrites the native observation into a fixed layout: four fields per tracked host, ordered by host slot, with an episode-level context block appended.

Distinguishes it from: The encoder, which comes after it and is fitted. This step resolves structural mismatch, meaning that the two environments now produce vectors of the same shape with the same field in the same position. It does not resolve what the numbers in those positions mean.

Operational test: Are the two observation vectors the same width with the same field order? That is what this step delivers, and the CyberBattleSim result is the demonstration that it is not sufficient.

In my project: The four fields are named on the page and in the methodology section in the same words. The context block carries the previous action and the previous reward.

the kill-chain intent interface

“a shared abstraction layer that constructs ϕ and ψ without modifying the underlying simulators”

Definition: The two mappings taken together, plus the shared representation they meet in. It sits between the two simulators, with the observation mapping on one side and the action mapping on the other.

Distinguishes it from: Either mapping alone. The interface is the claim that the kill chain is the right shared vocabulary, and both mappings inherit that claim.

Operational test: Name the shared thing. If the answer is a phase ordering that both environments can express, the interface is doing its job. If one environment cannot express a phase, that phase is where the mapping goes constant.

In my project: The central block of the architecture figure, and the reason the related work section reaches for domain-adversarial alignment: cyber offense follows a shared kill-chain progression, and most simulators capture it.

phase injection

Definition: The wrapper’s per-slot phase counter, kept where a simulator exposes only terminal compromise flags and no intermediate kill-chain phase. The counter advances as the corresponding native actions succeed, and its value is written into the observation before the encoder sees it.

Distinguishes it from: An observed phase, which the source environment supplies directly. The injected value is an approximation reconstructed from action outcomes, and the discussion names it as one of the four limits on the current results.

Operational test: Ask whether the target reports the field or whether the wrapper is counting. If the wrapper is counting, the field is synthetic and any claim resting on it inherits that.

In my project: It is what makes the phase-ordered representation available in every target regardless of what the target natively tracks.

Two symbols, four jobs

The methodology section defines ϕ as the state mapping and ψ as the action mapping. It then names the decoder D_ψ and the discriminator C_ϕ , so both symbols carry a second job inside the same subsection.

The results section adds a third use. It says the winning condition applies ϕ -encoded observations, where what is meant is observations that went through the deterministic mapping *and* the fitted encoder E_θ . Read against the methodology’s own definition, ϕ -encoded would mean the feature-engineering condition, which is the condition that wins nothing on that target.

Nothing about the method is unclear once the tables are read. The notation is, and the fix is a rename rather than an argument.

1.3 The encoder

the distributional gap

“That handles structural mismatch but leaves a distributional gap, the same feature carries different numbers across simulators, because networks differ in size, discovery rates, and reward scales.”

Definition: What remains after the layouts match. The field is in the same position in both vectors and its values are drawn from different distributions, because the two environments differ in network size, in how fast discovery happens, and in how reward is scaled.

Distinguishes it from: Structural mismatch, which the wrappers close. These are two failures with two remedies, and the CyberBattleSim result is what shows they are genuinely two.

Operational test: Hold the field position fixed and compare the two value distributions. Same position, different distribution, is this gap.

In my project: Measured as the distance between the two domain centroids before and after encoding.

the DAPN encoder

“A lightweight encoder closes it, trained with adversarial domain confusion so a discriminator cannot tell which simulator an observation came from, with reconstruction to stop the encoder collapsing everything to one point, and a mean-alignment term to steady early training.”

website, transfer-to-realistic-environments.md

Definition: A shallow network with layer normalization, mapping the variable-distribution part of the aligned observation into a latent the policy reads. Three losses train it: adversarial, reconstruction, and mean alignment.

Distinguishes it from: The original DAPN, which is an image-based few-shot classifier. The adaptation keeps the domain-adversarial objective and replaces the backbone with a small network over kill-chain features.

Operational test: Is anything about the policy being changed? No. The encoder sits in front of a frozen policy, which is what keeps the setting zero-shot.

In my project: The third of the four evaluation conditions, and the one that carries the large-gap target.

adversarial domain confusion

“A binary discriminator C_ϕ predicts which simulator an observation came from; the encoder is trained to fool it.”

RAISE, 3_methodology.tex

Definition: The alignment objective. A discriminator is trained to name the source environment from the latent, and the encoder is trained against it, in both directions symmetrically.

Distinguishes it from: A distance penalty between the two distributions, which the mean-alignment term supplies separately. The adversarial term is what removes the domain signal the mean term cannot see.

Operational test: Can a classifier still name the environment from the latent? Chance accuracy is the target and the stopping rule.

In my project: The reason the encoder can be trained without target rewards at all: the label it needs is which simulator produced the observation, which is free.

reconstruction against latent collapse

“A paired decoder D_ψ reconstructs the input from the latent, preventing the encoder from collapsing all observations to a single point under adversarial pressure.”

RAISE, 3_methodology.tex

Definition: The second loss, and the guard on the first. Under adversarial pressure alone the cheapest way to confuse a discriminator is to output the same vector for everything, which aligns the domains and destroys the observation.

Distinguishes it from: An autoencoder objective in its own right. Nothing here wants a good reconstruction. It wants the latent to remain able to distinguish observations, and reconstruction is the cheapest way to require that.

Operational test: Turn the term off and look at the spread of the target latents. A point mass is the failure this term exists to prevent, and the discussion reports it as observed rather than as feared.

In my project: Applied to both domains. Reconstructing only the source is the variant that collapses the target.

mean alignment

“ $\mathcal{L}_{\text{align}}$ minimizes the distance between source and target latent means, stabilizing early training.”

RAISE, 3_methodology.tex

Definition: The third loss. It pulls the two latent centroids together directly, which supplies a gradient in the epochs before the discriminator has converged and has anything useful to say.

Distinguishes it from: The adversarial term, which acts on the whole distribution and is what the checkpoint criterion reads. This one acts on the first moment only and is a training aid.

Operational test: Does the term have anything to do after the discriminator is at chance? Very little, which is what makes it early-training support rather than an alignment objective in its own right.

In my project: The third term of the three-term objective, and the smallest claim of the three.

the bypass stream

“features that mean the same thing in both environments”

website, transfer-to-realistic-environments.md

“Routing them through adversarial alignment suppresses the goal signal and the agent stops behaving goal-directedly.”

website, transfer-to-realistic-environments.md

Definition: The split made before encoding. Features whose distribution varies across simulators go through the encoder. Features that already mean the same thing in both, which here are the flags identifying the goal, go around it and are concatenated back at the output.

Distinguishes it from: A feature-selection step. Nothing is dropped. What is decided is which features are allowed to be aligned, and the finding is that aligning the wrong ones is worse than

not aligning at all.

Operational test: For a candidate feature, ask whether its meaning or only its scale differs across the two environments. Only-scale goes through. Same meaning goes around.

In my project: The page calls it the detail that turned out to matter, which is the strongest thing it says about any single design decision. The mechanism is that adversarial pressure treats the goal flag as domain evidence and erases it.

random-policy pretraining

“Encoder pretraining needs only observations from a random policy in each environment. No trajectories, no rewards from the target.”

website, transfer-to-realistic-environments.md

Definition: The data requirement, stated as the whole of it. Observations collected by acting uniformly at random in each environment. No trained policy is needed in the target and no reward signal is read from it.

Distinguishes it from: Every transfer method that needs target-domain trajectories or target-domain returns. This is what makes the method available before a target policy exists, which is the deployment case.

Operational test: Ask what has to exist in the target before the encoder can be trained. If the answer is a running environment and nothing else, it is this.

In my project: Two thousand paired observations and fifty epochs, against two hundred thousand timesteps to train a policy from scratch. The discussion puts those two numbers side by side, and that comparison is the deployment argument.

1.4 The evaluation

the four conditions

Definition: The ladder the results are read along. A random policy as the lower bound. Transfer by index-aligned padding, with no shared schema and no encoder. Transfer through the two wrappers, which tests whether feature alignment alone suffices. And the same plus the encoder.

Distinguishes it from: An ablation of one system. Each step adds exactly one mechanism, so a failure localizes to the mechanism that was added, and the second condition exists to show what a policy does when the vectors merely have the same width.

Operational test: For any reported number, name which of the four produced it. The paired numbers only mean something as a ladder.

In my project: Run against both targets. The two targets return opposite rankings of the third and fourth conditions, which is the paper’s main finding.

no-op collapse

“Faced with this mismatch, the policy’s learned structure provides no usable signal: mean nodes owned collapses to 0.00 and mean return to 19, as the policy effectively emits only no-ops.”

RAISE, 6_experimentalresults.tex

“This failure is total, not graceful.”

RAISE, 7_discussion.tex

Definition: The failure mode of dimensional alignment without distributional alignment. The policy receives a vector of the right width whose values mean something else, and it stops acting.

Distinguishes it from: Degraded performance. A degraded policy still plays and scores badly. This one owns zero nodes, which is below the random baseline’s 2.60, and the shape of the failure is the finding.

Operational test: Look at the action distribution rather than the score. All no-ops is this, and it is diagnosable in one episode.

In my project: The CyberBattleSim result under feature engineering alone, and the reason the paper can claim the state distribution rather than the observation shape is what transfers.

the source-to-target gap

Definition: The difference between a condition’s win rate in the source and its win rate in the target, used as the measure of how much of a policy is source-specific.

Distinguishes it from: Target win rate alone, which cannot separate a policy that transfers well from a policy that was weak everywhere. The comparison here is between a condition that nearly solves the source and loses over half of it, and a condition that is weaker at home and holds most of what it has.

Operational test: Report both numbers or neither. A target win rate without its source counterpart does not support a transfer claim.

In my project: On NetSecGame, feature engineering runs 99.1% in the source and 47.4% in the target, a drop of 51.7 points. The encoder condition runs 60.5% and 45.2%, a gap of 15.3 points, which is a 70% reduction. This is the answer the paper gives to its second question.

centroid distance

Definition: The distance between the mean latent vectors of the two domains, used as the direct check that the alignment objective did what it says.

Distinguishes it from: Win rate, which measures the consequence. This measures the mechanism, and it is reported before any transfer result so that a reader can see the encoder worked as an encoder before seeing whether it helped as a policy input.

Operational test: Was the quantity measured on the latents rather than on the observations? Only the first is evidence about the encoder.

In my project: 68.44 before, 28.05 after, a 59% reduction, with the t-distributed projection of the two clouds beside it.

Jensen-Shannon divergence between action distributions

“Transferred policies behave more like the NASim emulator policy than the untransferred source does”

RAISE, 6_experimentalresults.tex

Definition: The behavioural similarity measure used where win rate is unavailable. The action-

type distribution over trajectories is compared against the distribution of a policy evaluated natively on the emulator.

Distinguishes it from: A performance metric. The untransferred source policy does not win on the emulator at all, so its behaviour is all that can be compared, and this measure compares behaviour without requiring success.

Operational test: Ask what the reference distribution is. Without a native policy on the target to compare against, the number has nothing to mean.

In my project: 0.087 for the translated source-to-emulator policy and 0.085 for the second translated policy, against 0.111 for the untransferred source. This is the whole of the answer to the third question, and the paper says so by calling the result plausibility rather than transfer.

transfer regimes

“feature engineering can match or exceed it when schemas align but fails outright when they diverge, whereas DAPN produces useful policies in both regimes”

RAISE, 6_experimentalresults.tex

Definition: The two-regime reading of the result. Where the source and target already share a feature schema, the translator alone is enough and the encoder adds nothing measurable. Where the schemas diverge, the translator alone wins nothing and the encoder wins everything.

Distinguishes it from: A claim that the encoder is better. On the narrow gap it is not, and the paper says so twice.

Operational test: Compare the two schemas first. If they already share a vocabulary, an encoder is a refinement step and should be priced as one.

In my project: The conclusion states it as the paper’s position, and the discussion draws the tooling consequence: operators working across environments that already share a feature vocabulary may not need an encoder at all.

win rate is a poor metric for the encoder

“win rate is a poor metric for evaluating state encoders when the action translation layer is itself sufficient for task completion”

RAISE, 7_discussion.tex

Definition: A methodological verdict on the project’s own headline number. When the translator alone completes the task, the encoder’s contribution is invisible to a binary success measure, and the near-tie on the narrow gap is a property of the metric rather than of the encoder.

Distinguishes it from: A null result. The claim is not that the encoder did nothing. It is that this measurement could not have detected it.

Operational test: Does the condition being compared against already win? Then the comparison has no headroom and a different measure is needed.

In my project: The discussion names the replacements: step efficiency, progress rate, and performance on harder instances, meaning more nodes required, tighter budgets, branching topologies.

The abstract and the table

The abstract reports roughly 42.2% win rates for policies whose source performance is roughly 60.5%. The NetSecGame table reports 45.2% for the condition whose source figure is 60.5%. The source number matches and the target number does not. Whichever is right, the two sentences describe the same row and have to agree before the paper is read by anyone who checks.

1.5 The environments

CyberWheel as the source

Definition: The environment every transferred policy is trained in. Chosen because a competent policy is reachable under a modest budget, because its kill-chain observation is compact with a high proportion of decision-relevant features, and because its hosts and subnets are configurable through a specification file.

Distinguishes it from: The targets, which are never trained in. Holding the source fixed is what makes the two target results comparable to each other.

Operational test: Ask which environment produced the policy weights. That one is the source, and there is only ever one.

In my project: The initial direction was the reverse, from CyberBattleSim into CyberWheel, and it was changed after poor results in favour of transferring out of the better-structured environment.

decision-relevance

Definition: The proportion of an observation's features that the agent's choice actually depends on. It is used as an environment property in its own right, alongside dimensionality.

Distinguishes it from: Observation dimension. Two environments in the comparison table share a dimension of 512 and sit at opposite ends of this property, which is the reason the property is listed separately.

Operational test: Would removing the feature change the best action in any reachable state? If not, it is dimension without relevance.

In my project: The stated reason CyberBattleSim is sample-inefficient to train in, and part of the reason it is the source that was rejected.

NetSecGame, the narrow gap

Definition: The target whose action and observation spaces are closer to the source. Its training cost is comparable to the source and its task is harder, requiring discovery followed by impact over a wider action set, which lengthens the horizon.

Distinguishes it from: The other target, on the axis the experiment cares about. Small gap, hard task, is the combination that makes it the case where the translator alone is competitive.

Operational test: Compare the two schemas, not the two scores. Shared vocabulary is what narrow means here.

In my project: Evaluated over three seeds and five hundred episodes per seed, under a twenty-five step budget with a random target.

CyberBattleSim, the large gap

Definition: The target whose observation space is substantially different and whose action space is only partially aligned with the source. High dimensional, weakly structured, and sample-inefficient to train in, with a short horizon once a policy exists.

Distinguishes it from: NetSecGame, again on the gap axis. Large gap, easy task, is the combination that produces the paper’s starkest table.

Operational test: Same test. Partially aligned actions and substantially different observations is what large means here.

In my project: The chain-12 configuration, twenty episodes, win at eight nodes owned. Random owns 2.60 nodes and wins nothing, the translator alone owns zero, and the encoder condition wins all twenty.

NASim emulation mode

“Evaluated across CyberWheel as the source, with NetSecGame and CyberBattleSim as targets at different distances from it”

website, [transfer-to-realistic-environments.md](#)

Definition: The deployment proxy. NASim runs in simulation and also supports an emulation mode where nodes are instantiated as containers or virtual machines, vulnerabilities are replicated on live services, and exploits are executed through Metasploit.

Distinguishes it from: The two simulator targets, by what an action does. A transfer into this mode is the closest thing in the project to the sim-to-real case, which is why the third question is asked against it and answered with a behavioural measure rather than a win rate.

Operational test: Does the action invoke a tool against a running service? Then the target is the emulation and not the simulation of the same environment.

In my project: The same environment supplies both modes, which is what makes the comparison between them clean. Full emulation results are held for the complete version of the work.

2 Working

the shared schema

Definition: The fixed-width vector the two wrappers write into: four fields per tracked host slot, ordered by slot, plus the episode-level context block. It is the thing both environments are made to speak.

Distinguishes it from: Either native observation space, which the policy never sees after the wrappers are in place.

the kill chain

Definition: The phase ordering the shared vocabulary is built on: network discovery, service

discovery, exploitation, post-exploitation, exfiltration, and a terminal phase. Both mappings are organized by it.

Distinguishes it from: A technique taxonomy. The chain is an ordering, and what the method needs from it is that the ordering means the same thing in every environment even where the primitives do not.

PPO

Definition: The algorithm the source policies are trained with. It appears in this file only as the thing that produced the weights, since nothing about the transfer mechanism depends on the learner.

index-aligned padding

Definition: The second condition's mechanism. The target observation is zero-padded or truncated to the source width, with no attempt to put the same field in the same place.

Distinguishes it from: The state alignment wrapper, which puts the same field in the same place. The gap between the two conditions is exactly what field-level correspondence buys.

the white-box assumption

Definition: The setting the current method operates in: the structure of the target's state and action spaces is known, because the wrapper has to be written against it.

Distinguishes it from: A setting in which only interaction traces from the target are available. The encoder already runs on traces alone. The wrappers do not, and that is where the assumption sits.

open-loop translation

Definition: The property of the current translation layer named in the discussion: it resolves an intent into a native action without reading back whether the action succeeded, so it does not adapt to observation feedback.

Distinguishes it from: A closed version that would condition the next resolution on the outcome of the last. The discussion lists this as one of the four limits on the current results.

Metasploit

Definition: The exploitation framework the emulation mode executes through. Its presence is what makes an action in that mode a tool invocation against a live service rather than a state update.

3 Peripheral

DAPN, the original

Definition: The domain-adversarial prototypical network the encoder is adapted from, an image-based few-shot classifier addressing the tension between domain invariance and per-class discriminativeness. The adaptation treats kill-chain stage as the semantic class.

domain-adversarial training

Definition: The family the method belongs to: learn a feature space that preserves semantic classes while a discriminator cannot recover the domain. The related work section places the work in it and says why it fits, which is that cyber offense follows a shared kill-chain progression.

the NP-hardness of the coarsest abstraction

Definition: The result cited for why sim2sim is not the easy case: even with both environments fully specified, identifying an abstraction that relates them is computationally hard. It is the technical support for the position that supplying the missing ground truth does not make the problem go away.

attack transferability

Definition: An adversarial input crafted against one model succeeding against a second model the attacker never had access to. Sense S1 in `dict_transfer_learning.tex`, and it occurs nowhere in this project.

Distinguishes it from: Every sense above, by the sign of the outcome. Here the moving is the goal. There the moving is the failure. A reader arriving from adversarial machine learning will read the paper's title against that sense first.

4 The clashes, listed

Three. Two are folded into the entries above and the third is a division of labour rather than a disagreement.

Term	The two uses	Where it shows
ϕ and ψ	The state and action mappings, in the methodology	The decoder and discriminator subscripts in the same subsection, and the encoder in the results
the win rate on Net-SecGame	42.2%, in the abstract	45.2%, in the table, for the row whose source figure the abstract quotes correctly
transfer	An offensive policy across simulators, here	A defensive policy across two backends of one testbed, in <code>td_training_in_realistic_environments.tex</code>

The first two are fixable by editing. The third is not a fault: the two projects sit under one heading on the site because they are two answers to one problem, and a reader meeting both on the same page needs to be told which noun the verb is taking. `dict_transfer_learning.tex` records the pair as senses P1 and P2 and says the two projects use them against each other, one removing the transfer step and one studying it.

5 What crosses into other files

The general senses of transfer are not defined here. Policy transfer across environments, sim-to-sim and sim-to-real as a named pair, generalization to unseen configurations, and the survey sense of transfer learning are all in `dict_transfer_learning.tex`, with the outside anchors that fix their boundaries. What this file adds is the vocabulary of one method: the two mappings, the encoder, the four conditions, and the environments.

One distinction from that file matters enough to repeat. What this method does is closer to unsupervised domain adaptation than to transfer learning in the survey sense, because the encoder sees unlabeled observations from the target and no reward from it, while the policy takes no gradient step in the target at all.

The realism vocabulary is not here. Whether the emulation proxy is realistic enough to support the sim-to-real claim being made on it is a question for the Metrion file, which answers it per claim rather than per environment. This file records the measurement and the reference distribution, and stops there.

6 Sources

- Every quoted sentence is from the website page `transfer-to-realistic- environments.md`, under `content/overview/3-year-agenda/ toward-deployment/`, from the Toward Deployment overview `README.md`, or from the RAISE manuscript. The locator on each entry says which.
- The problem statement, the two mappings, the encoder architecture and the three losses are from `Sections/3_methodology.tex`. The formal statement of the two sub-problems and the definition of both mappings are there.
- The regime placement, the siloed-per-simulator argument and the reason domain-adversarial alignment fits cyber offense are from `Sections/2_relatedwork.tex`.
- The environment table, the four environment descriptions, the source rationale and the initial transfer direction are from `Sections/4_env.tex`.
- The four conditions, the kill-chain phase mapping table, both result tables, the centroid distances and the three divergence figures are from `Sections/6_experimentalresults.tex`.
- The four named limits, the encoder cost comparison, the verdict on win rate as a metric and the tooling consequence are from `Sections/7_discussion.tex`.
- The two-regime statement, and the list of what emulation adds beyond the representational gap, are from `Sections/9_conclusion.tex`.
- The three tiers, the three fluency levels and the four fields are from the Tech Fluency page. The reason a term whose two uses disagree belongs inside its entry is in `lexicon_method.tex`, in this folder.